SEP
SECRETARÍA DE EDUCACIÓN PÚBLICA

INSTITUTO TECNOLÓGICO de la Costa Grande Zihuatanejo, Gro.

TECNOLÓGICO NACIONAL DE MÉXICO

NOMBRE DE LA ACTIVIDAD

U6_A2 Reporte Conectividad

PRESENTA

Infante Aguilar Luis Ángel -23570067

MATERIA

Taller de base de datos.

ASESOR

María Elza Quintana Contreras.

ZIHUATANEJO GUERRERO, MEXICO.

FECHA

09/09/2025

ÍNDICE DE CONTENIDO

ÍNDICE DE CONTENIDO i

ÍNDICE DE FIGURAS ii

ÍNDICE DE TABLAS iii

1 DRIVERS 5

1.1 Driver de ADO.NET 5

2 SCRIPTS. 13

2.1 Script de conexión de ADO.NET. 13

3 INTERFAZ IUX. 19

3.1 Interfaz de ADO.NET 19

4 OPERACIONES. 26

4.1 Operaciones en ADO.NET 26

5 REFERENCIAS BIBLIOGRÁFICAS 33

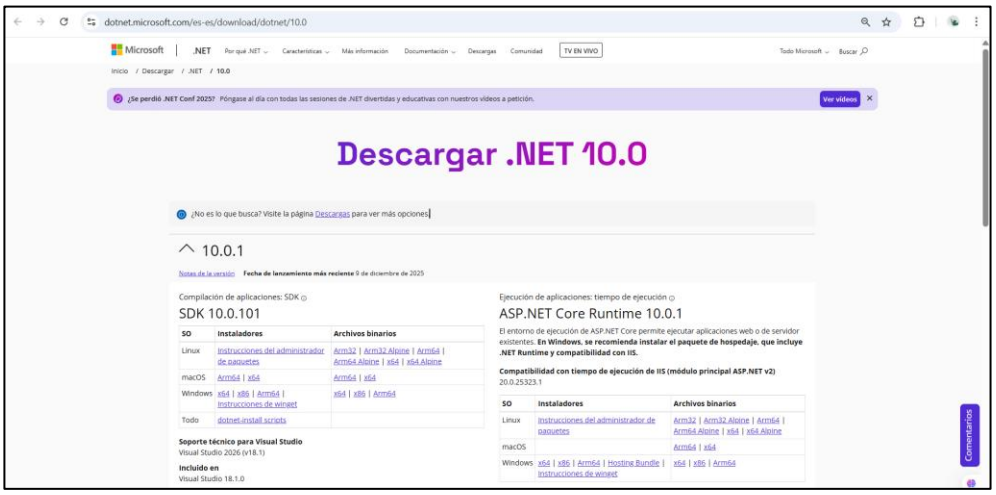
ÍNDICE DE FIGURAS

ÍNDICE DE TABLAS

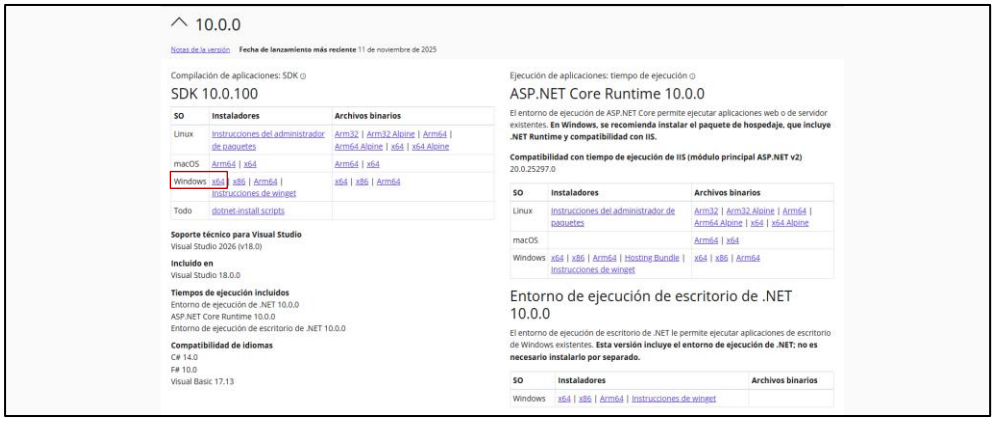
1 DRIVERS.

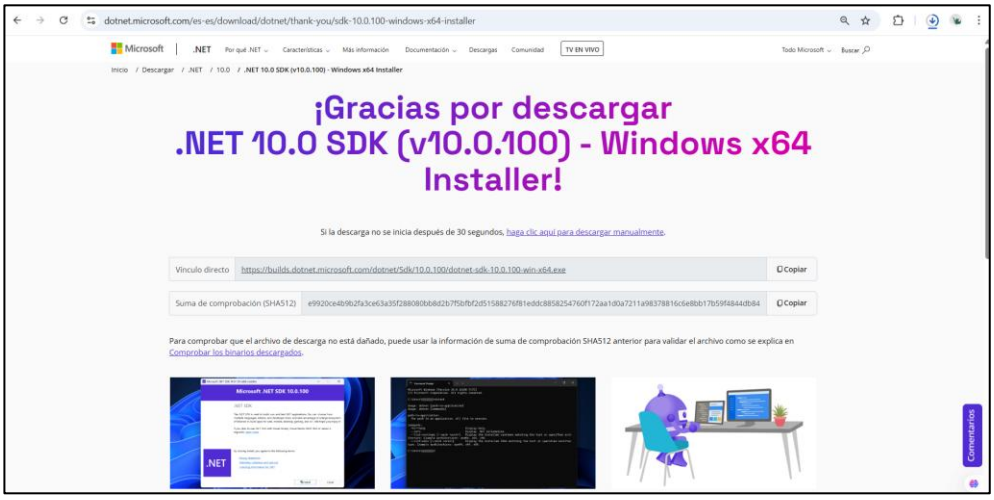
1.1 Driver de ADO.NET.

1. Abre https://dotnet.microsoft.com/download/dotnet/10.0

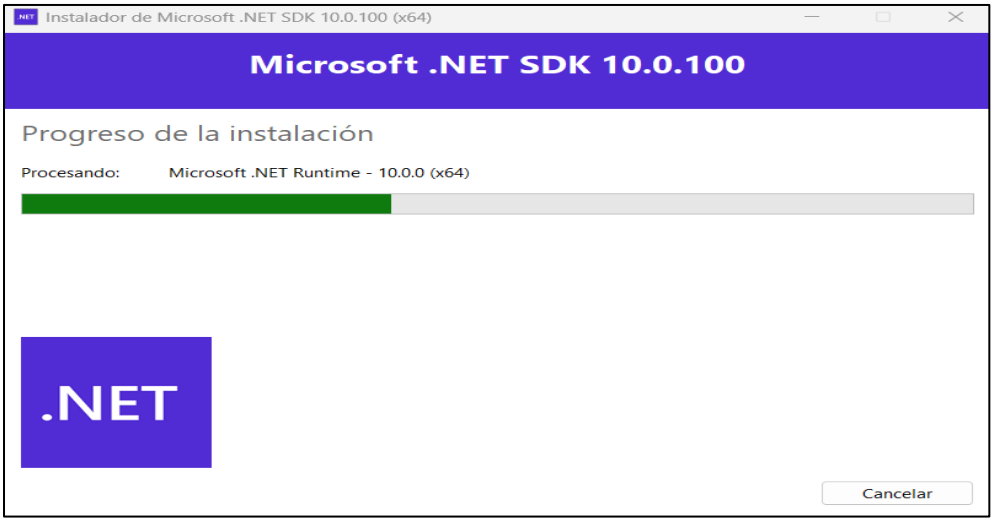


2. Descarga .NET 10.0 SDK (Windows x64).



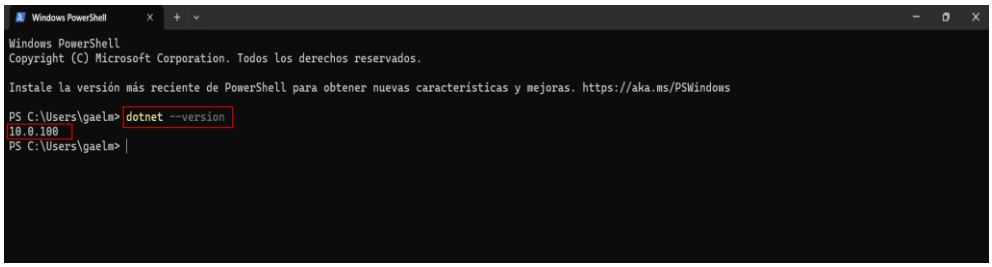


3. Ejecuta el instalador `.exe`.

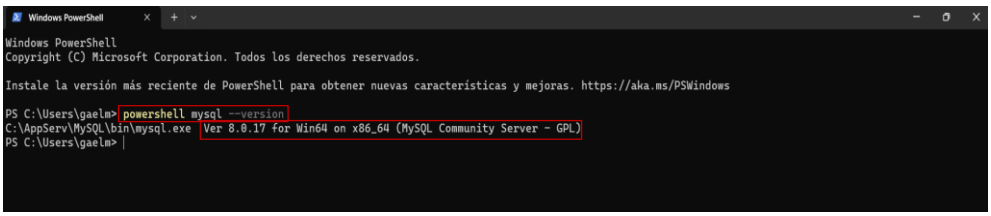


4. Sigue los pasos: Next → Finish.

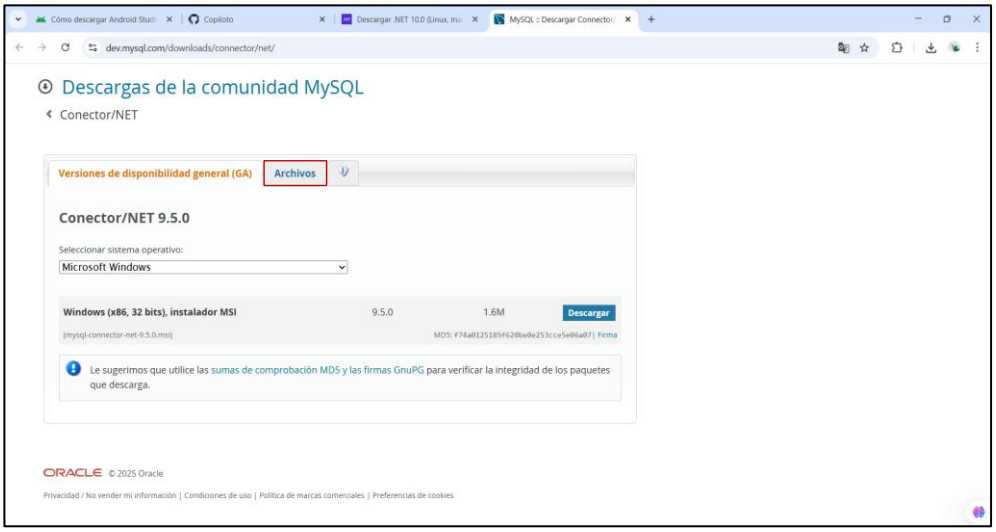
5. Abre PowerShell y ejecuta: powershell dotnet –version.

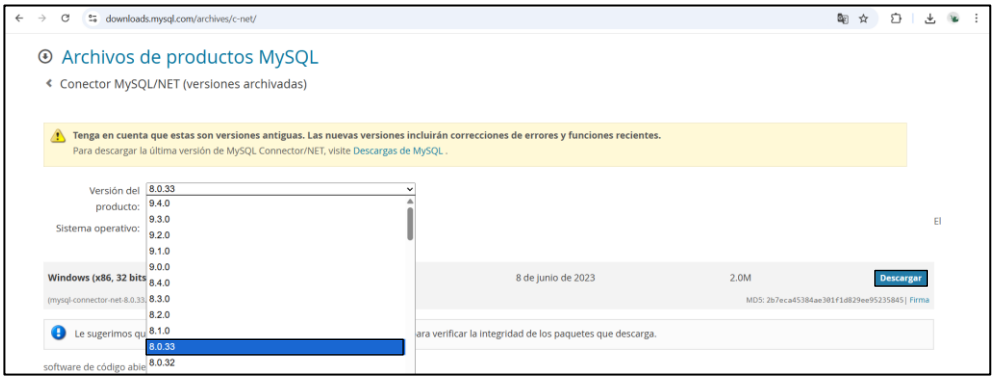


6. Abre PowerShell y ejecuta: ```powershell mysql –version.

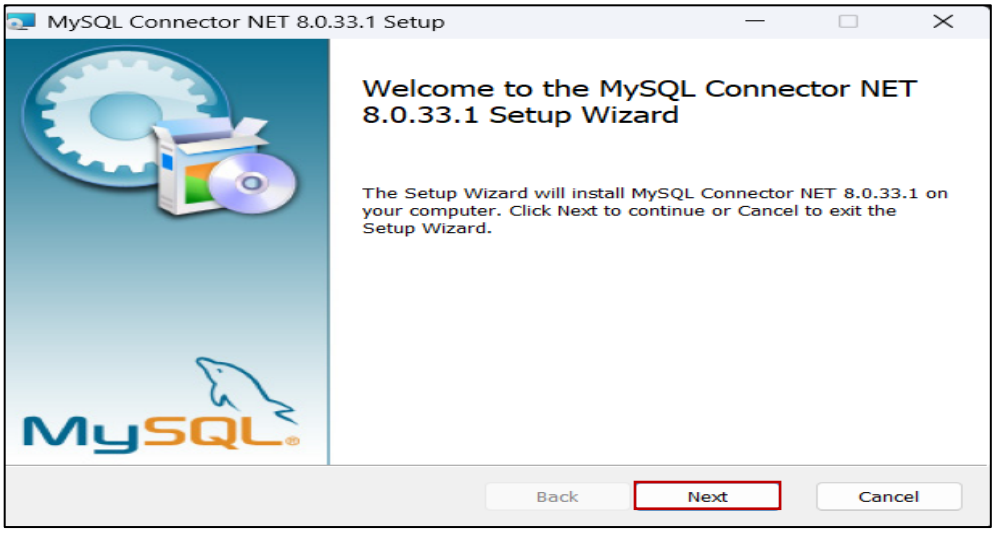


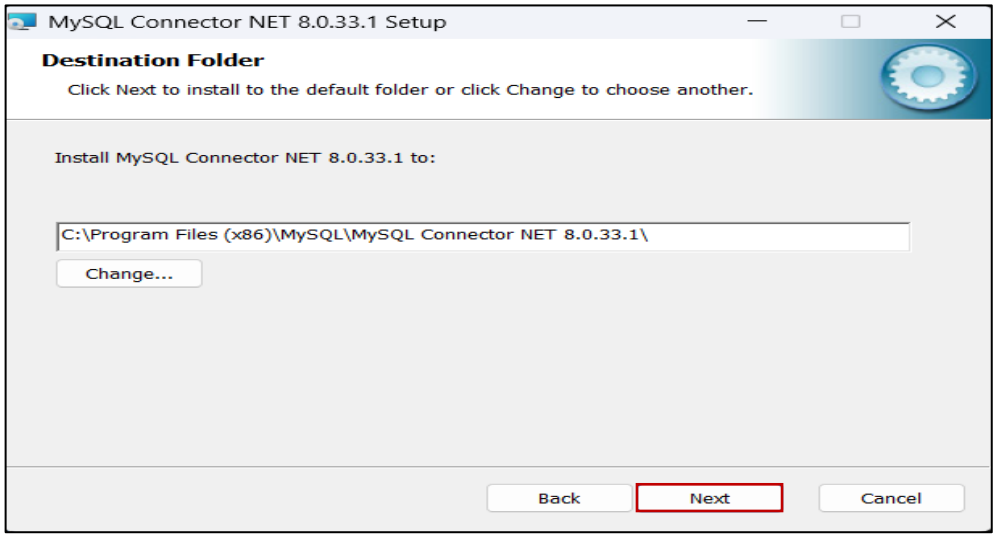
7. Descarga MySQL Connector/NET 8.0.33 (.msi para Windows).

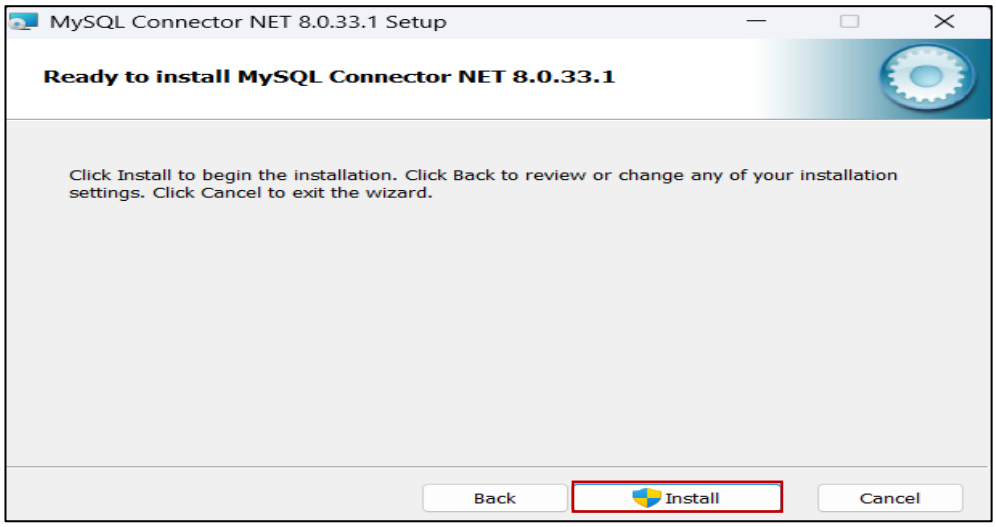


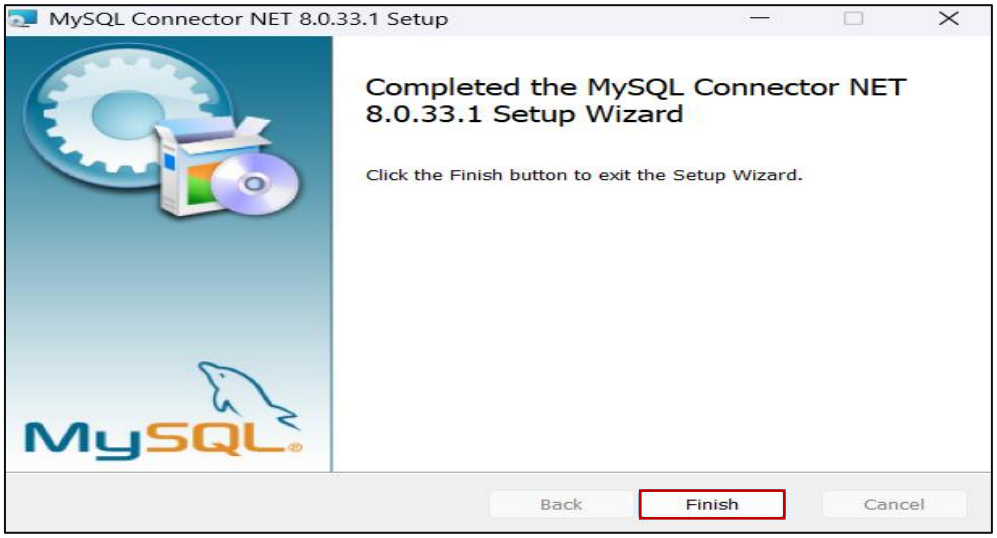


8. Ejecutar el archive previamente descargado.

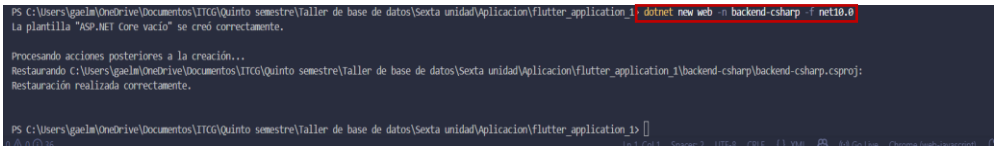


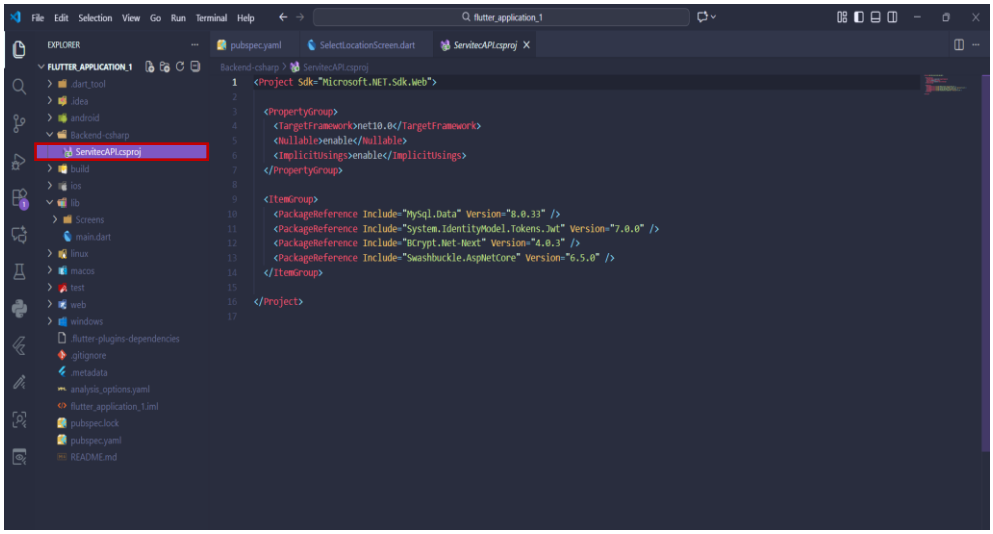






9. Creamos una carpeta dentro de la aplicación, la cual tendrá el backend de conexión ADO.NET C#, de igual manera creamos el archivo ServitecApi.csproj para incluir las dependencias de la aplicación, como el encriptamiento y el mysql.data, etc.





10. Dentro de VS code o en power Shell, ejecutaremos el siguiente comando “dotnet add package MySql.Data --version 8.0.33” dentro de la carpeta Backend-csharp.

```
PS C:\Users\gaelm\OneDrive\Documentos\ITCG\Quinto semestre\Taller de base de datos\Sexta unidad\Aplicacion\flutter_application_1> cd Backend-csharp
PS C:\Users\gaelm\OneDrive\Documentos\ITCG\Quinto semestre\Taller de base de datos\Sexta unidad\Aplicacion\flutter_application_1\Backend-csharp> dotnet add package MySql.Data --version 8.0.33
info : La validación de la cadena de certificados X.509 utilizará el almacén de confianza predeterminado seleccionado por .NET para la firma de código.
info : La validación de la cadena de certificados X.509 utilizará el almacén de confianza predeterminado seleccionado por .NET para la marca de tiempo.
info : Agregando PackageReference para el paquete "MySql.Data" al proyecto "C:\Users\gaelm\OneDrive\Documentos\ITCG\Quinto semestre\Taller de base de datos\Sexta unidad\Aplicacion\flutter_application_1
\Backend-csharp\ServitecAPI.csproj".
info : Restaurando paquetes para C:\Users\gaelm\OneDrive\Documentos\ITCG\Quinto semestre\Taller de base de datos\Sexta unidad\Aplicacion\flutter_application_1\Backend-csharp\ServitecAPI.csproj...
info :   GET https://api.nuget.org/v3-flatcontainer/system.identitymodel.tokens.jwt/index.json
info :   GET https://api.nuget.org/v3-flatcontainer/swashbuckle.aspnetcore/index.json
info :   GET https://api.nuget.org/v3-flatcontainer/bcrypt.net-next/index.json
info :   GET https://api.nuget.org/v3-flatcontainer/mysql.data/index.json
info :   OK https://api.nuget.org/v3-flatcontainer/swashbuckle.aspnetcore/index.json 312 ms
info :   GET https://api.nuget.org/v3-flatcontainer/swashbuckle.aspnetcore/6.5.0/swashbuckle.aspnetcore.6.5.0.nupkg
info :   OK https://api.nuget.org/v3-flatcontainer/system.identitymodel.tokens.jwt/index.json 345 ms
info :   GET https://api.nuget.org/v3-flatcontainer/system.identitymodel.tokens.jwt/7.0.0/system.identitymodel.tokens.jwt.7.0.0.nupkg
info :   OK https://api.nuget.org/v3-flatcontainer/bcrypt.net-next/index.json 370 ms
info :   OK https://api.nuget.org/v3-flatcontainer/swashbuckle.aspnetcore/6.5.0/swashbuckle.aspnetcore.6.5.0.nupkg 35 ms
info :   GET https://api.nuget.org/v3-flatcontainer/bcrypt.net-next/4.0.3/bcrypt.net-next.4.0.3.nupkg
info :   OK https://api.nuget.org/v3-flatcontainer/mysql.data/index.json 374 ms
info :   GET https://api.nuget.org/v3-flatcontainer/mysql.data/8.0.33/mysql.data.8.0.33.nupkg
info :   OK https://api.nuget.org/v3-flatcontainer/bcrypt.net-next/4.0.3/bcrypt.net-next.4.0.3.nupkg 26 ms
info :   OK https://api.nuget.org/v3-flatcontainer/mysql.data/8.0.33/mysql.data.8.0.33.nupkg 85 ms
info :   OK https://api.nuget.org/v3-flatcontainer/system.identitymodel.tokens.jwt/7.0.0/system.identitymodel.tokens.jwt.7.0.0.nupkg 244 ms
info :   GET https://api.nuget.org/v3-flatcontainer/google.protobuf/index.json
info :   GET https://api.nuget.org/v3-flatcontainer/k4os.compression.lz4.streams/index.json
info :   GET https://api.nuget.org/v3-flatcontainer/portable.bouncycastle/index.json
info :   GET https://api.nuget.org/v3-flatcontainer/swashbuckle.aspnetcore.swagger/index.json
info :   GET https://api.nuget.org/v3-flatcontainer/system.configuration.configurationmanager/index.json
info :   GET https://api.nuget.org/v3-flatcontainer/system.security.permissions/index.json
info :   GET https://api.nuget.org/v3-flatcontainer/swashbuckle.aspnetcore.swaggergen/index.json
info :   GET https://api.nuget.org/v3-flatcontainer/swashbuckle.aspnetcore.swaggerui/index.json
info :   GET https://api.nuget.org/v3-flatcontainer/microsoft.identitymodel.jsonwebtokens/index.json
info :   GET https://api.nuget.org/v3-flatcontainer/microsoft.extensions.apidescription.server/index.json
info :   GET https://api.nuget.org/v3-flatcontainer/microsoft.identitymodel.tokens/index.json
info :   OK https://api.nuget.org/v3-flatcontainer/portable.bouncycastle/index.json 131 ms
info :   GET https://api.nuget.org/v3-flatcontainer/portable.bouncycastle/1.9.0/portable.bouncycastle.1.9.0.nupkg
info :   OK https://api.nuget.org/v3-flatcontainer/portable.bouncycastle/1.9.0/portable.bouncycastle.1.9.0.nupkg 34 ms
```

11. ecutar el siguiente commando “dotnet restore”.

```
PS C:\Users\gaelm\OneDrive\Documentos\ITCG\Quinto semestre\Taller de base de datos\Sexta unidad\Aplicacion\flutter_application_1\Backend-csharp> dotnet restore
Restaurar correcto con 3 advertencias en 0.6s
    C:\Users\gaelm\OneDrive\Documentos\ITCG\Quinto semestre\Taller de base de datos\Sexta unidad\Aplicacion\flutter_application_1\Backend-csharp\ServitecAPI.csproj : warning NU1902: El paquete "Microso
ft.IdentityModel.JsonWebTokens" 7.0.0 tiene una vulnerabilidad de gravedad moderada conocida, https://github.com/advisories/GHSA-59j7-ghrg-fj52
    C:\Users\gaelm\OneDrive\Documentos\ITCG\Quinto semestre\Taller de base de datos\Sexta unidad\Aplicacion\flutter_application_1\Backend-csharp\ServitecAPI.csproj : warning NU1904: El paquete "System.
Drawing.Common" 4.7.0 tiene una vulnerabilidad de gravedad critica conocida, https://github.com/advisories/GHSA-rxg9-xrhp-64gj
    C:\Users\gaelm\OneDrive\Documentos\ITCG\Quinto semestre\Taller de base de datos\Sexta unidad\Aplicacion\flutter_application_1\Backend-csharp\ServitecAPI.csproj : warning NU1902: El paquete "System.
IdentityModel.Tokens.Jwt" 7.0.0 tiene una vulnerabilidad de gravedad moderada conocida, https://github.com/advisories/GHSA-59j7-ghrg-fj52

Compilación correcto con 3 advertencias en 0.9s
PS C:\Users\gaelm\OneDrive\Documentos\ITCG\Quinto semestre\Taller de base de datos\Sexta unidad\Aplicacion\flutter_application_1\Backend-csharp>
```

12. Configuramos el archivo appsettings.jason que se creó anteriormente y colocamos los datos correspondientes a nuestro mysql, usuario, server, localhost, id, contraseña y el puerto en donde está habilitado el mysql.

```
backend-csharp > {} appsettings.json > AllowedHosts
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=localhost;Database=servitec;Uid=root;Pwd=LU2040#G;Port=3306;"
  },
  "JWT": {
    "Secret": "tu_clave_secreta_super_segura_aqui_1234567890",
    "ExpiryDays": 30
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*"
}
```

13. Modificamos el archivo program.cs para agregar los servicios a utilizar, cargar la configuración de tipo JSON y ver el estado del servidor ADO.NET.

```
using ServitecAPI.Services;
var builder = WebApplication.CreateBuilder(args);

// Cargar configuración
builder.Configuration
    .AddJsonFile("appsettings.json", optional: false, reloadOnChange: true)
    .AddEnvironmentVariables();
// Agregar servicios
builder.Services.AddControllers();
builder.Services.AddScoped<DatabaseService>();
builder.Services.AddScoped<AuthService>();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();
// Agregar CORS
builder.Services.AddCors(options =>
{
    options.AddPolicy("AllowAll", b =>
    {
        b.AllowAnyOrigin()
         .AllowAnyMethod()
         .AllowAnyHeader();
    });
});

var app = builder.Build();

// Configurar middleware
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();
app.UseCors("AllowAll");
app.UseAuthorization();
app.MapControllers();

Console.WriteLine("Servidor Servitec (C# ADO.NET) corriendo en puerto 3000");
Console.WriteLine("URL: http://localhost:3000");
Console.WriteLine("Health check: http://localhost:3000/api/health");
app.Run("http://0.0.0.0:3000");
```

13. Ejecutamos la aplicación con el comando “dotnet run”.

```
PS C:\Users\gaelm\OneDrive\Documentos\ITCG\Quinto semestre\Taller de base de datos\Sexta unidad\Aplicacion\flutter_application_1\Backend-csharp> dotnet run
Usando la configuración de inicio de C:\Users\gaelm\OneDrive\Documentos\ITCG\Quinto semestre\Taller de base de datos\Sexta unidad\Aplicacion\flutter_application_1\Backend-csharp\Properties\launchSettin
gs.json...
Compilando...
C:\Users\gaelm\OneDrive\Documentos\ITCG\Quinto semestre\Taller de base de datos\Sexta unidad\Aplicacion\flutter_application_1\Backend-csharp\ServitecApi.csproj : warning NU1902: El paquete "Microsoft.I
El paquete "Microsoft.IdentityModel.JsonWebTokens" 7.0.0 tiene una vulnerabilidad de gravedad moderada conocida, https://github.com/advisories/GHSA-59j7-ghrg-fj52
C:\Users\gaelm\OneDrive\Documentos\ITCG\Quinto semestre\Taller de base de datos\Sexta unidad\Aplicacion\flutter_application_1\Backend-csharp\ServitecApi.csproj : warning NU1904: El paquete "System.Draw
ing.Common" 4.7.0 tiene una vulnerabilidad de gravedad crítica conocida, https://github.com/advisories/GHSA-rxg9-xrhp-64gj
C:\Users\gaelm\OneDrive\Documentos\ITCG\Quinto semestre\Taller de base de datos\Sexta unidad\Aplicacion\flutter_application_1\Backend-csharp\ServitecApi.csproj : warning NU1902: El paquete "System.Iden
tityModel.Tokens.Jwt" 7.0.0 tiene una vulnerabilidad de gravedad moderada conocida, https://github.com/advisories/GHSA-59j7-ghrg-fj52
C:\Users\gaelm\OneDrive\Documentos\ITCG\Quinto semestre\Taller de base de datos\Sexta unidad\Aplicacion\flutter_application_1\Backend-csharp\ServitecApi.csproj : warning NU1902: El paquete "Microsoft.I
dentityModel.JsonWebTokens" 7.0.0 tiene una vulnerabilidad de gravedad moderada conocida, https://github.com/advisories/GHSA-59j7-ghrg-fj52
C:\Users\gaelm\OneDrive\Documentos\ITCG\Quinto semestre\Taller de base de datos\Sexta unidad\Aplicacion\flutter_application_1\Backend-csharp\ServitecApi.csproj : warning NU1904: El paquete "System.Draw
ing.Common" 4.7.0 tiene una vulnerabilidad de gravedad crítica conocida, https://github.com/advisories/GHSA-rxg9-xrhp-64gj
C:\Users\gaelm\OneDrive\Documentos\ITCG\Quinto semestre\Taller de base de datos\Sexta unidad\Aplicacion\flutter_application_1\Backend-csharp\ServitecApi.csproj : warning NU1902: El paquete "System.Iden
tityModel.Tokens.Jwt" 7.0.0 tiene una vulnerabilidad de gravedad moderada conocida, https://github.com/advisories/GHSA-59j7-ghrg-fj52
Servidor Servitec (C# ADO.NET) corriendo en puerto 3000
URL: http://localhost:3000
Health check: http://localhost:3000/api/health
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://0.0.0.0:3000
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
      Content root path: C:\Users\gaelm\OneDrive\Documentos\ITCG\Quinto semestre\Taller de base de datos\Sexta unidad\Aplicacion\flutter_application_1\Backend-csharp
```

2 SCRIPTS.

2.1 Script de conexión de ADO.NET.

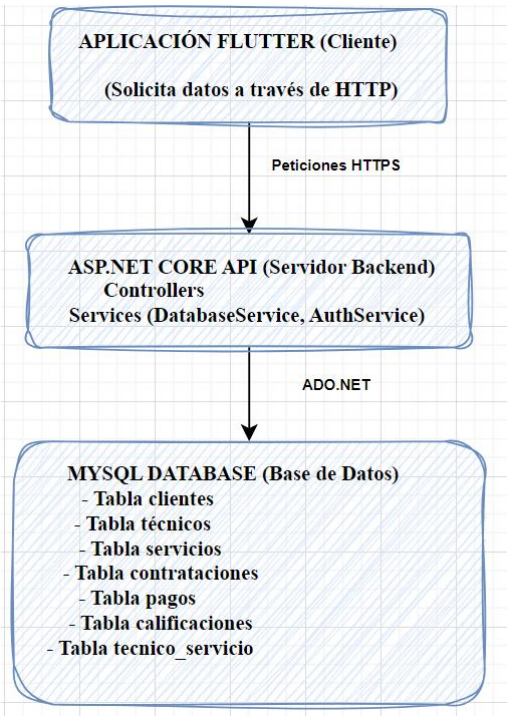
El script de conexión ADO.NET es el componente encargado de establecer y gestionar la comunicación entre la aplicación backend (desarrollada en C# con ASP.NET Core) y la base de datos MySQL.

ADO.NET (ActiveX Data Objects .NET) es una tecnología de Microsoft que permite:

- Conectarse a bases de datos relacionales.
- Ejecutar comandos SQL.
- Manipular datos de manera segura.
- Proteger contra inyecciones SQL mediante parámetros.

En este proyecto, se utiliza el Proveedor MySQL (MySql.Data.MySqlClient) para conectarse a la base de datos SERVITEC.

Arquitectura de la conexión.



1. Archivo de configuración creado anteriormente appsettings.json, previo al módulo de conexión, aquí se ingresan las credenciales de la base de datos que serán cargados por el módulo de conexión.

   Este archivo permite:

   - No mezclar datos sensibles con lógica.
   - Sin recompilar la aplicación.
   - Desarrollo, prueba, producción.
   - Credenciales en archivo de configuración, no en código.

```
{
  "ConnectionStrings": {
    "DefaultConnection":
"Server=localhost;Database=servitec;Uid=root;Pwd=LU2040#G;Port=3306;"
  },
  "JWT": {
    "Secret": "tu_clave_secreta_super_segura_aqui_1234567890",
    "ExpiryDays": 30
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*"
}
```

2. La clase ServicioDatos es el Servicio de Acceso a Datos (DAL - Data Access Layer) que encapsula toda la lógica de conexión y ejecución de consultas a la base de datos MySQL. Es la clase puente entre los controladores (Controllers) de la API y la base de datos, garantizando que todas las operaciones de BD se realicen de forma segura, asincrónica y mediante parámetros. Métodos incluidos en la clase servicioDatos:

   - Directivas de uso: Las directivas de uso al inicio del archivo permiten usar clases y métodos de espacios de nombres externos sin escribir el nombre completo cada vez. El namespace organiza la clase dentro del proyecto.

- Variable cadena_conexion: almacena la cadena de conexión a la base de datos. El modificador readonly indica que solo se puede asignar una vez (en el constructor) y no puede cambiar después. Esto es una medida de seguridad.
- Constructor: Es un método especial que se ejecuta cuando se crea una instancia de la clase. En este caso, recibe un objeto IConfiguration (inyectado por ASP.NET Core) que contiene toda la configuración del proyecto, incluyendo la cadena de conexión del archivo appsettings.json.
- Obtener conexión: Es un módulo que crea una **nueva instancia de conexión MySQL cada vez que es llamado.
- Método de encriptación: Las contraseñas jamás deben almacenarse en texto plano en la base de datos. Se deben encriptar usando un algoritmo seguro como BCrypt.

  Este módulo proporciona dos métodos:
  - Cifrar Contraseña: Recibe una contraseña en texto plano, la encripta usando BCrypt, devuelve el hash (irreversible) y se usa cuando un usuario se registra.
  - Verificar Contraseña: Recibe una contraseña ingresada por el usuario, recibe el hash almacenado en la BD, verifica sí coinciden y se usa cuando el usuario intenta login.
- Ejecutar scalar async: Este método ejecuta consultas SELECT que devuelven un solo valor. Es decir, una celda individual de la base de datos.
- Ejecutar comando async: Este método ejecuta comandos que modifican datos en la base de datos, pero no devuelven filas. Se usa para: Agregar nuevos registros(insert), Modificar registros existentes (update) y Eliminar registros (delete).
- Ejecutar consulta async: Este es el método más versátil. Ejecuta consultas select que devuelven múltiples filas y las convierte en una estructura flexible: List<Dictionary<string, object>>.

```
using MySql.Data.MySqlClient;
using System.Security.Cryptography;
using System.Text;

namespace ServitecAPI.Services
{
    public class ServicioDatos
    {
        // ====== VARIABLE PRIVADA ======
        // Almacena la cadena de conexión a la base de datos
        private readonly string _cadenaConexion;

        // ====== CONSTRUCTOR ======
        // Recibe la configuración del proyecto y extrae la cadena de conexión
        public ServicioDatos(IConfiguration config)
        {
            _cadenaConexion = config.GetConnectionString("DefaultConnection")
                ?? throw new InvalidOperationException("Cadena de conexión no encontrada.");
        }

        // ====== MÉTODO: OBTENER CONEXIÓN ======
        // Crea una nueva instancia de conexión MySQL
        public ConexionMySQL ObtenerConexion()
        {
            return new ConexionMySQL(_cadenaConexion);
        }

        // ====== MÉTODO: CIFRAR CONTRASEÑA ======
        // Encripta una contraseña de texto plano usando BCrypt
        public string CifrarContraseña(string contraseña)
        {
            return BCrypt.Net.BCrypt.HashPassword(contraseña);
        }

        // ====== MÉTODO: VERIFICAR CONTRASEÑA ======
        // Verifica que una contraseña ingresada coincida con su hash
        public bool VerificarContraseña(string contraseña, string hash)
        {
            return BCrypt.Net.BCrypt.Verify(contraseña, hash);
        }

        // ====== MÉTODO: EJECUTAR SCALAR (VALOR ÚNICO) ======
        // Ejecuta una consulta que devuelve UN SOLO valor
        public async Task<T?> EjecutarScalarAsync<T>(
            string consulta,
```

```
        Dictionary<string, object>? parametros = null)
    {
        using (var conexion = ObtenerConexion())
        {
            await conexion.AbrirAsync();
            using (var comando = new ComandoMySQL(consulta, conexion))
            {
                if (parametros != null)
                {
                    foreach (var param in parametros)
                    {
                        comando.Parametros.AgregarConValor($"@{param.Key}", param.Value);
                    }
                }
                var resultado = await comando.EjecutarScalarAsync();
                return resultado != null ? (T)Convert.ChangeType(resultado, typeof(T)) : default;
            }
        }
    }

    // ====== MÉTODO: EJECUTAR COMANDO (INSERT, UPDATE, DELETE) ======
    // Ejecuta comandos que modifican datos sin devolver resultados
    public async Task<int> EjecutarComandoAsync(
        string consulta,
        Dictionary<string, object>? parametros = null)
    {
        using (var conexion = ObtenerConexion())
        {
            await conexion.AbrirAsync();
            using (var comando = new ComandoMySQL(consulta, conexion))
            {
                if (parametros != null)
                {
                    foreach (var param in parametros)
                    {
                        comando.Parametros.AgregarConValor($"@{param.Key}", param.Value);
                    }
                }
                return await comando.EjecutarAsync();
            }
        }
    }
    // ====== MÉTODO: EJECUTAR CONSULTA (SELECT) ======
    // Ejecuta consultas que devuelven múltiples filas
    public async Task<List<Dictionary<string, object>>> EjecutarConsultaAsync(
```

```
            string consulta,
            Dictionary<string, object>? parametros = null)
        {
            var resultados = new List<Dictionary<string, object>>();
            using (var conexion = ObtenerConexion())
            {
                await conexion.AbrirAsync();
                using (var comando = new ComandoMySQL(consulta, conexion))
                {
                    if (parametros != null)
                    {
                        foreach (var param in parametros)
                        {
                            comando.Parametros.AgregarConValor($"@{param.Key}", param.Value);
                        }
                    }
                    using (var lector = await comando.EjecutarLectorAsync())
                    {
                        while (await lector.LeerAsync())
                        {
                            var fila = new Dictionary<string, object>();
                            for (int i = 0; i < lector.ConteoColumnas; i++)
                            {
                                fila[lector.ObtenerNombre(i)] = lector.ObtenerValor(i);
                            }
                            resultados.Add(fila);
                        }
                    }
                }
            }
            return resultados;
        }
    }
}
```

3 INTERFAZ IUX.

3.1 Interfaz de la aplicación con flutter de ADO.NET.

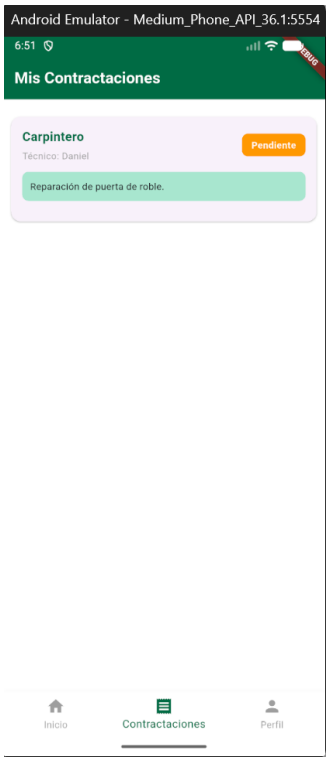
1. Pantalla de bienvenida a la aplicación.

2. Registro de clientes.

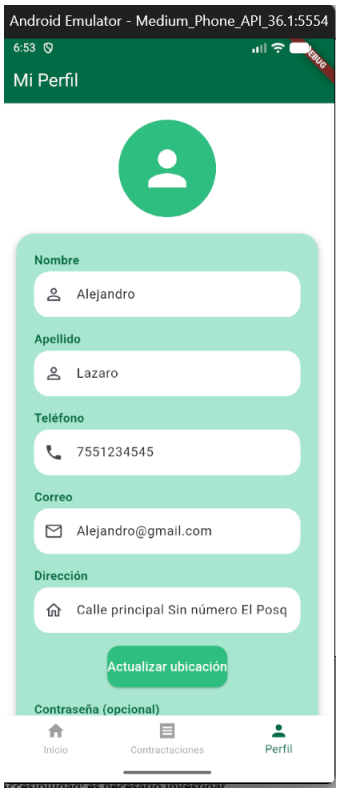
3. Pestaña de inicio al cliente, donde puede buscar técnico por nombre o buscar al técnico según las cajas de categorías.

4. Solicitar contratación a un técnico.

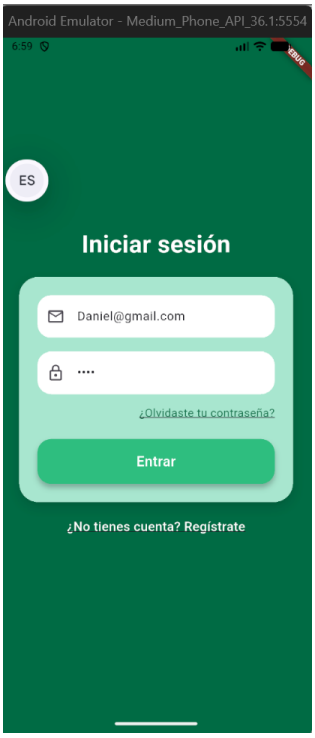
5. Pantalla para visualizar y dar seguimiento a las contrataciones.



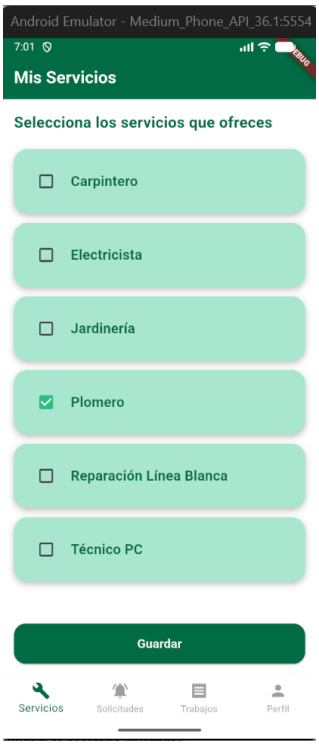
6. Pantalla perfil, donde el cliente puede consultar y actualizar sus datos.



7. Pantalla inicio de sesión, para login de cliente o de técnico.

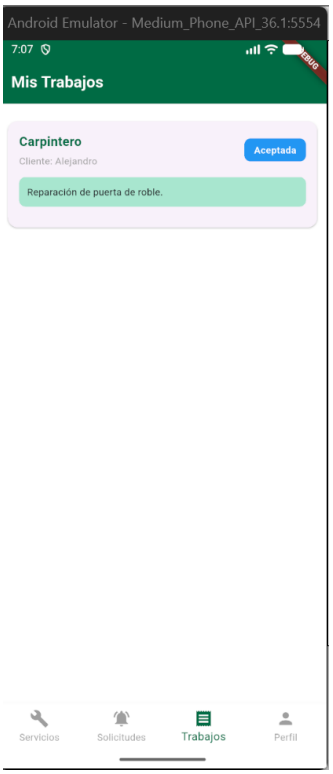


8. Pantalla home de técnico, en la sección servicios donde el técnico puede agregar los tipos de servicio que ofrece.

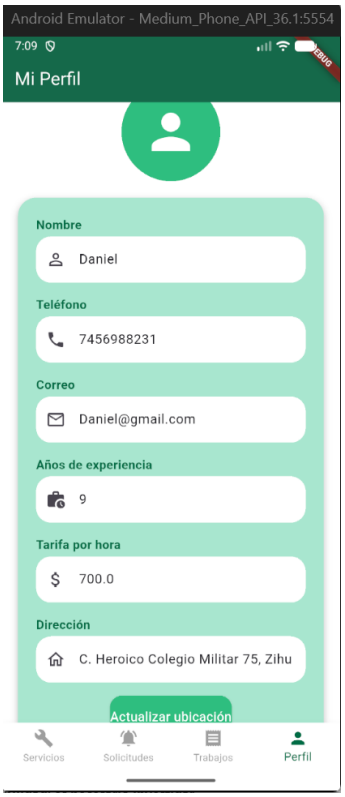


9. Pantalla donde el técnico recibe las solicitudes y las acepta o rechaza.

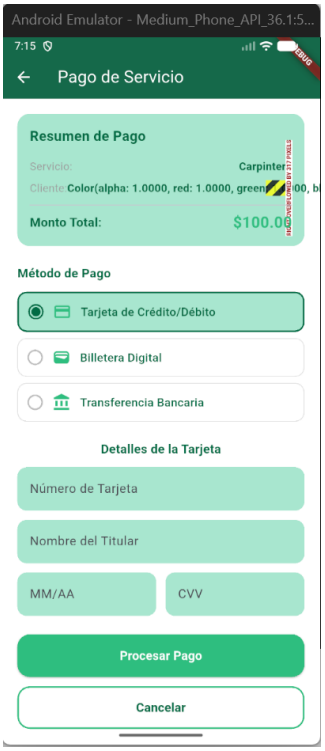
10. Pantalla trabajos, donde se da seguimiento a las solicitudes de contratación aceptadas.



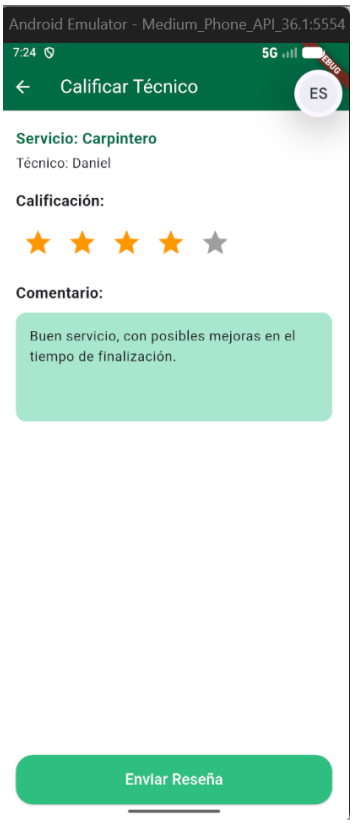
11. Pantalla perfil donde el técnico puede consultar y actualizar sus dados de registro.



12. Pantalla de pago que se le habilita al cliente una vez que el técnico acepte su contratación.



13. Pantalla de calificación que se habilita en cliente una vez que el técnico finalice el servicio.



4 OPERACIONES.

4.1 Operaciones en ADO.NET

- INSERT.

a) Registrar cliente: Se usa cuando un nuevo cliente crea cuenta en la app y es de rol Público (sin necesidad de token JWT).

```
        [HttpPost("registrar/cliente")]
    public async Task<IActionResult> RegistrarCliente([FromBody] SolicitudRegistroCliente solicitud)
    {
        try
        {

            // Verifica que el email no esté registrado
            var clienteExistente = await _bd.ExecuteScalarAsync<int>(
                "SELECT COUNT(*) FROM clientes WHERE email = @email",
                new Dictionary<string, object> { { "email", solicitud.Email } }
            );

            if (clienteExistente > 0)
                return Conflict(new { error = "Email ya está registrado." });

            // Encripta la contraseña usando BCrypt
            var hashContrasena = _bd.EncriptarContrasena(solicitud.Contrasena);

            // Insertar nuevo cliente y obtener su ID
            var consulta = @"
            INSERT INTO clientes (nombre, apellido, email, hash_contrasena, telefono,
direccion_texto,latitud, longitud)
                VALUES (@nombre, @apellido, @email, @hash, @telefono, @direccion_texto, @lat, @lng);
                SELECT LAST_INSERT_ID();
            ";

            var parametros = new Dictionary<string, object>
            {
                { "nombre", solicitud.Nombre },
                { "apellido", solicitud.Apellido },
                { "email", solicitud.Email },
                { "hash", hashContrasena },
                { "telefono", solicitud.Telefono },
                { "direccion_texto", solicitud.TextoDireccion },
                { "lat", solicitud.Latitud },
```

```
                { "lng", solicitud.Longitud }
            };

            var idCliente = await _bd.ExecuteScalarAsync<int>(consulta, parametros);

            if (idCliente <= 0)
                return StatusCode(500, new { error = "Error al crear el cliente." });

            // Genera token JWT para autenticación
            var token = _autenticacion.GenerarToken(idCliente, solicitud.Email, "cliente");

            return Ok(new
            {
                token,
                tipo_usuario = "cliente",
                id_usuario = idCliente,
                email = solicitud.Email,
                nombre = solicitud.Nombre
            });
        }
        catch (Exception ex)
        {
            Console.WriteLine($"Error en RegistrarCliente: {ex.Message}");
            Console.WriteLine($"Stack: {ex.StackTrace}");
            return StatusCode(500, new { error = ex.Message });
        }
    }
```

- UPDATE:

  a) Actualizar datos del cliente: Construye la consulta para actualizar dinámicamente según los campos enviados del cliente.

```
        public async Task<IActionResult> ActualizarPerfilCliente(int id, [FromBody] ActualizarSolicitudCliente
solicitud)
        {
            try
            {
                // Construye la query dinámicamente según los campos enviados
                var actualizaciones = new List<string>();
                var parametros = new Dictionary<string, object> { { "id", id } };

                if (!string.IsNullOrEmpty(solicitud.Nombre))
                {
                    actualizaciones.Add("nombre = @nombre");
                    parametros["nombre"] = solicitud.Nombre;
                }
                if (!string.IsNullOrEmpty(solicitud.Apellido))
                {
                    actualizaciones.Add("apellido = @apellido");
                    parametros["apellido"] = solicitud.Apellido;
                }
                if (!string.IsNullOrEmpty(solicitud.Email))
                {
                    actualizaciones.Add("email = @email");
                    parametros["email"] = solicitud.Email;
                }
                if (!string.IsNullOrEmpty(solicitud.Telefono))
                {
                    actualizaciones.Add("telefono = @telefono");
                    parametros["telefono"] = solicitud.Telefono;
                }
                if (!string.IsNullOrEmpty(solicitud.TextoDireccion))
                {
                    actualizaciones.Add("direccion_texto = @direccion_texto");
                    parametros["direccion_texto"] = solicitud.TextoDireccion;
                }
                if (solicitud.Latitud.HasValue)
                {
                    actualizaciones.Add("latitud = @latitud");
                    parametros["latitud"] = solicitud.Latitud;
                }
```

```
            if (solicitud.Longitud.HasValue)
            {
                actualizaciones.Add("longitud = @longitud");
                parametros["longitud"] = solicitud.Longitud;
            }
            if (!string.IsNullOrEmpty(solicitud.Contrasena))
            {
                actualizaciones.Add("hash_contrasena = @hash_contrasena");
                parametros["hash_contrasena"] = _bd.EncriptarContrasena(solicitud.Contrasena);
            }

            if (actualizaciones.Count == 0)
                return BadRequest(new { error = "No hay campos para actualizar" });

            actualizaciones.Add("actualizado_en = NOW()");

            // Actualizar cliente con campos dinámicos
            var consulta = $"UPDATE clientes SET {string.Join(", ", actualizaciones)} WHERE id_cliente = @id";
            await _bd.ExecuteNonQueryAsync(consulta, parametros);

            return Ok(new { mensaje = "Cliente actualizado exitosamente" });
        }
        catch (Exception ex)
        {
            return StatusCode(500, new { error = ex.Message });
        }
    }
}
```

b) Actualizar el estado de las contrataciones: Actualiza el estado de las contrataciones cliente según la aceptación o rechazo por parte del técnico.

```
        public async Task<IActionResult> ActualizarEstadoContratacion(int id, [FromBody]
ActualizarEstadoContratacionSolicitud solicitud)
        {
            try
            {
                // Estados válidos
                var estadosValidos = new[] { "Pendiente", "Aceptada", "En Progreso", "Completada", "Cancelada" };
                if (!estadosValidos.Contains(solicitud.Estado))
                    return BadRequest(new { error = "Estado inválido" });
```

```
                // Actualizar estado de la contratación
                var consulta = "UPDATE contrataciones SET estado = @estado, actualizado_en = NOW() WHERE
id_contratacion = @id";
                await _bd.ExecuteNonQueryAsync(consulta,
                    new Dictionary<string, object> { { "estado", solicitud.Estado }, { "id", id } });

                return Ok(new { mensaje = "Contratación actualizada", estado = solicitud.Estado });
            }
            catch (Exception ex)
            {
                return StatusCode(500, new { error = ex.Message });
            }
        }
```

- DELETE.

  a) Actualizar servicios del técnico: Para actualizar el servicio de un técnico y que sea del mismo tipo, elimina el servicio anterior con el id del técnico.

```
public async Task<IActionResult> ActualizarServiciosTecnico(int id, [FromBody] ActualizarServiciosTecnicoSolicitud
solicitud)
        {
            try
            {
                // Verifica que el técnico existe
                var tecnicoExiste = await _bd.ExecuteScalarAsync<int>(
                    "SELECT COUNT(*) FROM tecnicos WHERE id_tecnico = @id",
                    new Dictionary<string, object> { { "id", id } }
                );
                if (tecnicoExiste == 0)
                    return NotFound(new { error = "Técnico no existe" });

                //  Eliminar servicios anteriores
                await _bd.ExecuteNonQueryAsync(
                    "DELETE FROM tecnico_servicio WHERE id_tecnico = @id",
                    new Dictionary<string, object> { { "id", id } }
                );

                // Insertar nuevos servicios
                if (solicitud.IdsServicios != null && solicitud.IdsServicios.Count > 0)
                {
                    foreach (var idServicio in solicitud.IdsServicios)
                    {
                        await _bd.ExecuteNonQueryAsync(
                            "INSERT INTO tecnico_servicio (id_tecnico, id_servicio) VALUES (@idTecnico,
@idServicio)",
                            new Dictionary<string, object> { { "idTecnico", id }, { "idServicio", idServicio } }
                        );
                    }
                }

                return Ok(new { mensaje = "Servicios actualizados exitosamente" });
            }
            catch (Exception ex)
            {
                return StatusCode(500, new { error = ex.Message });
            }
        }
```

- SELECT.

    a) Obtener contrataciones: Consulta todas las contrataciones disponibles.

```
        public async Task<IActionResult> ObtenerContratacion(int id)
    {
        try
        {
            // Obtener contratación por ID
            var resultados = await _bd.ExecuteQueryAsync(
                "SELECT * FROM contrataciones WHERE id_contratacion = @id",
                new Dictionary<string, object> { { "id", id } }
            );

            if (resultados.Count == 0)
                return NotFound();

            return Ok(resultados[0]);
        }
        catch (Exception ex)
        {
            return StatusCode(500, new { error = ex.Message });
        }
    }
}
```

5 REFERENCIAS BIBLIOGRÁFICAS.

Zapatero, C. (2025, 3 septiembre). *Cómo configurar MySQL en Windows paso a paso*. ADSLZone. https://www.adslzone.net/esenciales/windows-10/instalar-mysql/

MICROSOFT, (2017). *Establecer la conexión - ADO.NET*. Microsoft Learn. https://learn.microsoft.com/es-es/dotnet/framework/data/adonet/establishing-the-connection

Martínez, G. A. (2023b, octubre 16). *Qué es ADO.NET y cómo utilizarlo*. Brave Developer. https://bravedeveloper.com/2023/10/15/que-es-ado-net/

Authors, M. (2010). *High-Performance .NET MySQL Driver - MySqlConnector*. https://mysqlconnector.net/